

Министерство науки и высшего образования Российской Федерации
РГУ нефти и газа (НИУ) имени И.М. Губкина

Кафедра безопасности информационных технологий

Лабораторная работа №6 по дисциплине
«Управление жизненным циклом информационных систем»
Вариант №1

Выполнил студент
группы КН-23-10
Вотинцева А.С.

Проверил д.т.н.,
профессор
Корнеев Н.В.

Москва 2026

Цель работы

Получить навыки конструирования систем с элементами ИИ на базе паттерна Python Data Science в SaaS Replit.

Общее задание

1. Разверните паттерн Python Data Science для работы над проектами в области науки о данных на Relit. Назовите реплику своей фамилией.
2. Выписать в отчет название уже интегрированных в паттерн библиотек, системных модулей и системных зависимостей и дать краткую характеристику назначения.
3. Запустите реплику и ознакомьтесь с кодом в файле `main.py`.
4. Закомментируйте код в строках 1-6. Продолжая работу в `main.py` с использованием паттерна Python Data Science напишите код для микросервиса решающего задачу кластеризации данных методом К-средних (KMeans). Используйте `make_blobs` функцию, доступную в `sklearn` пакете Python. Используйте `fit` функцию, доступную в `sklearn` пакете Python для поиска оптимального количества кластеров методом локтя. Если данные плохо кластеризованы используйте Elbow или Silhouette.

Микросервис должен:

- выводить в консоль набор данных: распределенных по одному нормально распределенному кластеру точек; прогнозируемые кластеры для каждой точки данных; координаты центроидов для каждого кластера; внутрикластерную сумму квадратов; количество итераций методом К-средних для получения минимальной суммы квадратов внутри кластера; размер каждого кластера;
- выводить в графическом интерфейсе визуализацию кластеров в виде диаграммы рассеяния; обозначение кластеров, используя предсказанные метки классов.

5. Разберите код микросервиса на функциональные части, назовите каждую часть в соответствии с выполняемым функционалом, например блок ввода данных, блок преобразования данных и т.п. в коде комментарием. Дайте

комментарии в каждой функциональной части по основным строкам кода в которых содержатся наиболее важные функции и/или методы.

6. Выполните пункт 5 в отношении конфигурационных файлов: .replit и replit.nix

Результаты выполнения общей части задания

Алгоритм выполнения задания:

1. Развернула паттерн Python Data Science для работы над проектами в области науки о данных на Relit. Назвала реплику Votintseva_LR6.

2. Выписала в отчет название уже интегрированных в паттерн библиотек, системных модулей и системных зависимостей и дала краткую характеристику назначения. Результаты представлены в таблице 1.

Таблица 1 - Итегрированные в паттерн библиотеки

Библиотека	Краткое описание
certifi	Набор корневых сертификатов для проверки SSL-соединений. Обеспечивает безопасные HTTPS-запросы.
charset-normalizer	Определение и нормализация кодировок текста. Помогает корректно читать файлы с неизвестной кодировкой.
contourpy	Вычисление и построение контурных линий (изолиний). Используется matplotlib для контурных графиков.
cycler	Управление последовательностями стилей в matplotlib (цвета, маркеры, линии).
fonttools	Работа со шрифтами (чтение, редактирование, преобразование форматов TTF/OTF).
idna	Поддержка международных доменных имён (IDNA) для работы с не-ASCII символами в URL.
joblib	Оптимизированная сериализация (сохранение/загрузка) больших Python-объектов (модели ML, массивы).
kiwisolver	Решатель системы линейных ограничений (Cassowary) для автоматического позиционирования элементов графиков.
matplotlib	Базовая библиотека для построения графиков: диаграммы рассеяния, линейные графики, гистограммы, сохранение в файлы.
mpmath	Вычисления с произвольной точностью (арифметика с плавающей точкой, комплексные числа, спецфункции).
numpy	Работа с многомерными массивами и матрицами. Высокопроизводительные математические операции, линейная алгебра.
packaging	Утилиты для работы с версиями пакетов (сравнение, парсинг). Используется pip и инструментами сборки.
pandas	Анализ и обработка табличных данных. Структуры DataFrame и Series, фильтрация, группировка, объединение.
pillow	Работа с растровыми изображениями: открытие, редактирование, изменение размера, сохранение в форматах.
pyarsing	Создание синтаксических анализаторов (парсеров). Используется matplotlib для разбора строк с формулами.

python-dateutil	Расширенная работа с датами и временем: парсинг, часовые пояса, относительные интервалы.
requests	Простая и удобная библиотека для выполнения HTTP-запросов (GET, POST и др.). Работа с веб-API.
scikit-learn	Машинное обучение: кластеризация (KMeans), классификация, регрессия, генерация данных (make_blobs), предобработка.
scipy	Научные вычисления: оптимизация, интегрирование, интерполяция, линейная алгебра, обработка сигналов, статистика.
seaborn	Высокоуровневая статистическая визуализация (построена на matplotlib). scatterplot, boxplot, heatmap, автоматическая раскраска кластеров.
six	Обеспечение совместимости кода между Python 2 и Python 3.
sympy	Символьная математика: алгебраические преобразования, решение уравнений, дифференцирование, интегрирование.
threadpoolctl	Управление количеством потоков в BLAS-библиотеках (Intel MKL, OpenBLAS). Ограничение параллельных вычислений.
urllib3	Мощный HTTP-клиент с поддержкой пула соединений, повторных запросов и SSL. Используется requests.

3. Запустила реплику и ознакомилась с кодом в файле main.py.

4. Закомментировала код в строках 1-6. Продолжая работу в main.py с использованием паттерна Python Data Science написала код для микросервиса решающего задачу кластеризации данных методом К-средних (KMeans). Использовала make_blobs функцию, доступную в sklearn пакете Python, fit функцию, доступную в sklearn пакете Python для поиска оптимального количества кластеров методом локтя.

5. Разобрала код микросервиса на функциональные части, назвала каждую часть в соответствии с выполняемым функционалом, например блок ввода данных, блок преобразования данных и т.п. в коде комментарием. Дала комментарии в каждой функциональной части по основным строкам кода в которых содержатся наиболее важные функции и/или методы.

6. Выполнила пункт 5 в отношении конфигурационных файлов: .replit и replit.nix

7. Фрагменты программного кода с комментариями выгружены в репозиторий GitHub и представлены по ссылке: https://github.com/lesyavotintseva/Votintseva_lr6.git

Алгоритм работы микросервиса кластеризации данных методом К-средних:

1. Генерация данных – создаётся синтетический набор из 200 точек с 2 признаками, распределённых по 4 гауссовым кластерам.

2. Определение оптимального числа кластеров. Для значений k от 1 до 12 рассчитывается внутрикластерная сумма квадратов. Оптимальное k определяется в точке изгиба графика. Результат сохраняется в файл chart14.png.

3. Обучение модели K-Means – модель обучается с оптимальными параметрами: `n_clusters=4`, `init='k-means++'`. На выходе получаются: метки кластеров для каждой точки, координаты центроидов, внутрикластерная сумма квадратов (`inertia`), количество итераций для сходимости.

4. Анализ размеров кластеров – подсчитывается количество точек, попавших в каждый кластер.

5. Визуализация кластеров – строятся две диаграммы рассеяния: без центроидов (chart15.png) – только точки с цветовой маркировкой по кластерам; с центроидами (chart16.png) – точки + центроиды в виде красных крестов.

6. Вывод результатов – все данные выводятся в консоль, графики сохраняются в файлы и отображаются на экране.

Блок схема кода представлена на рисунке 1.

Индивидуальное задание

1. Напишите код для микросервиса решающего задачу кластеризации данных методом К-средних (KMeans). Выполнить задание последовательно по пунктам 4-6 общего задания. Данные для кластеризации представляют собой массив M , который надо сгенерировать по варианту №1:

U записей должно быть 3 параметра. Первый изменяется в диапазоне $[0,1; 5]$, второй параметр – в диапазоне $[0,1; 3]$, третий параметр может принимать значения – 10%, 20%, 80%, 90%.

2. Задание выполняется каждым студентом отдельно. Для программного средства по варианту индивидуального задания из ЛР1, согласно ТЗ, выполнить следующие задания:

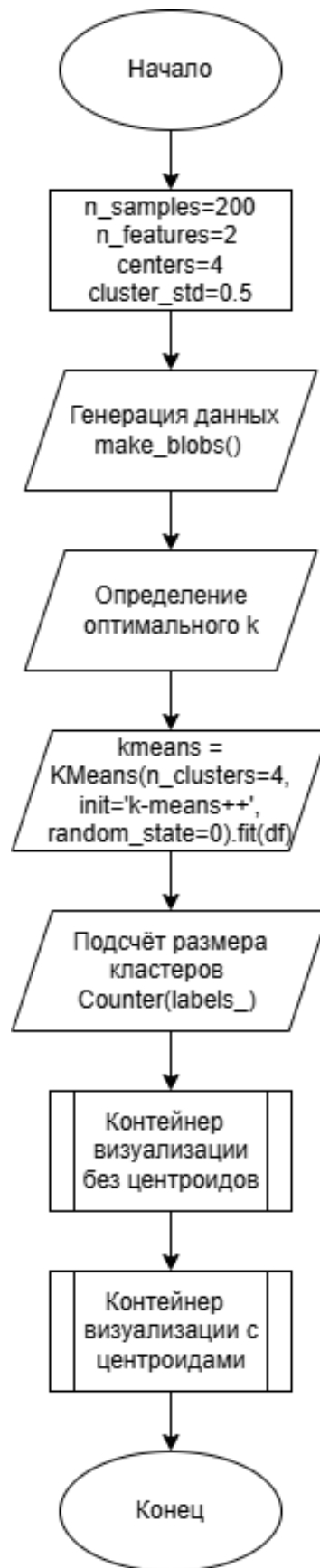


Рисунок 1 – Блок-схема к общему заданию

2. Задание выполняется каждым студентом отдельно. Для программного средства по варианту индивидуального задания из ЛР1, согласно ТЗ, выполнить следующие задания:

1. Создайте новую реплику. Назовите реплику своей фамилией. Напишите в реплике код контейнера для обеспечения безопасности.

2. Опишите структуру папок Вашего проекта контейнера для обеспечения безопасности. Раскройте содержание папок и основных файлов проекта.

3. Разберите код основных файлов Вашего проекта контейнера для обеспечения безопасности на функциональные части, назовите каждую часть в соответствии с выполняемым функционалом, например блок ввода данных, блок преобразования данных и т.п. в коде комментарием. Дайте комментарии в каждой функциональной части по основным строкам кода в которых содержатся наиболее важные функции и/или методы.

4. Выполните пункт 3 в отношении конфигурационных файлов: .replit и replit.nix.

5. Подготовьте комплект документации на созданный контейнер по ЕСПД.

Результаты выполнения индивидуальной части задания

Алгоритм выполнения задания 1:

1. Написала код для микросервиса решающего задачу кластеризации данных методом К-средних (KMeans) в соответствии с исходными данными по варианту 1.

2. Продолжая работу в main.py с использованием паттерна Python Data Science написала код для микросервиса решающего задачу кластеризации данных методом К-средних (KMeans). Использовала make_blobs функцию, доступную в sklearn пакете Python, fit функцию, доступную в sklearn пакете Python для поиска оптимального количества кластеров методом локтя.

3. Разобрала код микросервиса на функциональные части, назвала каждую часть в соответствии с выполняемым функционалом, например блок

ввода данных, блок преобразования данных и т.п. в коде комментарием. Дала комментарии в каждой функциональной части по основным строкам кода в которых содержатся наиболее важные функции и/или методы.

4. Выполнила пункт 3 в отношении конфигурационных файлов: .replit и replit.nix

5. Фрагменты кода с комментариями выгружены в репозиторий GitHub и представлены по ссылке: https://github.com/lesyavotintseva/Votintseva_lr6.git

Алгоритм работы микросервиса:

1. Инициализация и импорт библиотек – подключаются необходимые библиотеки.

2. Генерация данных по варианту 1 – создаётся синтетический набор из 200 записей с 3 параметрами: равномерное распределение в диапазоне [0,1; 5,0], равномерное распределение в диапазоне [0,1; 3,0], категориальные значения (10%, 20%, 80%, 90%), которые преобразуются в числовые метки (0, 1, 2, 3) для работы алгоритма K-means.

3. Вывод информации о данных в консоль – отображаются первые 10 строк набора данных, статистическое описание числовых параметров, распределение категорий третьего параметра.

4. Определение оптимального количества кластеров (метод локтя) – для значений k от 1 до 10 рассчитывается внутрикластерная сумма квадратов. Оптимальное количество кластеров определяется в точке изгиба графика. График метода локтя сохраняется в файл individual_elbow.png.

5. Обучение модели K-means – модель обучается с оптимальным количеством кластеров (определённым на шаге 4) и параметрами: init='k-means++', n_init=10. На выходе получаются: метки кластеров для каждой точки данных, координаты центроидов в трёхмерном пространстве, внутрикластерная сумма квадратов (inertia), количество итераций для сходимости алгоритма.

6. Анализ размеров кластеров – с помощью Counter() подсчитывается количество точек, попавших в каждый кластер, вычисляется процентное соотношение.

7. Вывод результатов кластеризации в консоль – отображаются: прогнозируемые кластеры для первых 20 точек, координаты центроидов для каждого кластера, внутрикластерная сумма квадратов, количество итераций, размер каждого кластера в абсолютных и относительных значениях.

8. 3D-визуализация кластеров – строятся две трёхмерные диаграммы рассеяния: без центроидов (левый график) – только точки данных с цветовой маркировкой по кластерам; с центроидами (правый график) – точки данных + центроиды в виде красных звёзд (*). На обоих графиках: ось X – параметр 1 (диапазон 0,1-5,0), ось Y – параметр 2 (диапазон 0,1-3,0), ось Z – параметр 3 с подписями категорий (10%, 20%, 80%, 90%). Результат сохраняется в файл individual_clusters_3d.png.

9. Кросс-анализ категорий по кластерам – с помощью pd.crosstab() строится таблица, показывающая, как распределились категории третьего параметра (10%, 20%, 80%, 90%) по полученным кластерам. Это позволяет оценить влияние категориального признака на кластеризацию.

10. Вывод итоговой статистики – в консоль выводятся: общее количество точек, количество признаков, диапазоны параметров 1 и 2, уникальные значения параметра 3.

Блок схема кода представлена на рисунке 2.



Рисунок 2 - Блок-схема к индивидуальному заданию

Алгоритм выполнения задания 2:

1. Создала новую реплику. Назовите реплику своей фамилией. Написала в реплике код контейнера для обеспечения безопасности при угрозе форсированного веб-браузинга согласно ТЗ из ЛР1.

2. Описала структуру папок проекта контейнера для обеспечения безопасности.

Корневая директория: artifacts/waf-container/

1. Директория waf_project/ – Django-проект (настройки)

Содержит конфигурационные файлы Django-проекта:

– __init__.py – файл-маркер, указывающий Python, что директория является пакетом

– settings.py – основная конфигурация Django: настройки базы данных, middleware, параметры логирования

– urls.py – корневая маршрутизация URL-адресов (обработка путей /waf/ и /admin/)

– wsgi.py – WSGI-точка входа для запуска приложения на production-сервере

2. Директория waf/ – основное Django-приложение (WAF-логика)

Содержит всю бизнес-логику web-application firewall:

– __init__.py – маркер пакета

– apps.py – конфигурация приложения, инициализация движка защиты (ProtectionEngine)

– middleware.py – middleware-компонент для перехвата каждого входящего HTTP-запроса и его проксирования

– protection.py – движок обнаружения атак (класс ProtectionEngine)

– models.py – модели базы данных: заблокированные IP-адреса, события безопасности, логи запросов, доверенные IP

– views.py – view-функции: веб-дашборд и 8 REST API-эндпоинтов

– urls.py – URL-маршруты приложения (префикс /waf/...)

– admin.py – регистрация моделей в панели администратора Django

- logger.py – JSON-форматтер для логов, совместимый с ELK-стеком

Поддиректории waf/:

- migrations/ – миграции базы данных SQLite

- 0001_initial.py – начальная миграция, создающая все WAF-модели

- templates/waf/ – HTML-шаблоны для веб-дашборда

- dashboard.html – главная страница дашборда с графиками (Chart.js, live-обновление)

- events.html – таблица событий безопасности

- blocked_ips.html – список заблокированных IP-адресов

- management/commands/ – кастомные команды для управления через manage.py

- seed_demo.py – заполнение базы демонстрационными данными (500 запросов, 150 событий)

3. Директория config/

Содержит файлы конфигурации:

waf_config.yaml – YAML-файл с настройками: пороги срабатывания, режим работы, белый список IP, URL защищаемого бэкенда

4. Директория logs/

security_events.json – файл со структурированными JSON-логами событий безопасности (совместим с ELK-стеком)

5. Директория .replit-artifact/

artifact.toml – файл регистрации сервиса в проху-сервере Replit (указывает порт 8000 и путь /waf)

6. Корневые файлы

- manage.py – основная точка входа для выполнения команд Django CLI

- requirements.txt – список зависимостей Python: Django, requests, PyYAML, whitenoise

- Dockerfile – инструкции для сборки Docker-образа контейнера (базовый образ python:3.11-slim)

- docker-compose.yml – файл оркестрации для запуска WAF-контейнера вместе с защищаемым приложением (nginx)
- db.sqlite3 – файл базы данных SQLite (хранит блокировки, события, логи)
- .gitignore – список файлов и папок, исключаемых из системы контроля версий Git

Раскрыла основных файлов проекта. Результаты представлены в таблице 2.

Таблица 2 - Ключевые файлы по функциям

Файл	Роль
middleware.py	Точка входа — каждый запрос проходит здесь; проверяет блокировку, проксирует на backend
protection.py	Мозг WAF — скользящее окно, счётчики уникальных URL, оценка порогов, вызов блокировки
models.py	Персистентность — хранит все блокировки, события и лог запросов в SQLite
views.py	Дашборд + REST API для управления (блок/разблок IP, статистика, симуляция атаки)
dashboard.html	Визуализация — Chart.js, автообновление каждые 10 сек через /waf/api/stats/
waf_config.yaml	Конфигурация без перекодирования: пороги, режим block/monitoring/disabled, whitelist
logger.py	Каждое событие безопасности пишется в JSON-файл в формате для ELK-стека

3. Разобрала код основных файлов проекта контейнера для обеспечения безопасности на функциональные части, назала каждую часть в соответствии с выполняемым функционалом, например блок ввода данных, блок преобразования данных и т.п. в коде комментарием. Дала комментарии в каждой функциональной части по основным строкам кода в которых содержатся наиболее важные функции и/или методы.

4. Выполнила пункт 3 в отношении конфигурационных файлов: .replit и replit.nix.

5. Код контейнера с комментариями выгрузила в репозиторий GitHub, представлен по ссылке: https://github.com/lesyavotintseva/Votintseva_WAF.git

6. Подготовила комплект документации на созданный контейнер по ЕСПД. Представлен в Приложении А.

Алгоритм работы контейнера для обеспечения безопасности (WAF)

Общее описание

Контейнер реализует функциональность Web Application Firewall (WAF), который перехватывает входящие HTTP-запросы, анализирует их на предмет признаков аномальной активности и принимает решение о блокировке или проксировании запроса к защищаемому бэкенд-приложению. Алгоритм работает на основе скользящего временного окна и трёх детекторов атак.

Пошаговое описание алгоритма

1. Перехват запроса в Middleware

При поступлении HTTP-запроса middleware-компонент перехватывает его и извлекает ключевые параметры: реальный IP-адрес клиента (из заголовка X-Forwarded-For при работе за прокси), запрашиваемый URL, HTTP-метод и User-Agent. Это необходимо для корректной идентификации клиента при работе в облачной среде.

2. Фильтрация системных путей

Запросы к служебным путям самого WAF (например, /waf/, /admin/, /static/) пропускаются без анализа. Это предотвращает блокировку легитимного доступа к панели управления и статическим файлам самого защитного механизма.

3. Проверка белого списка (whitelist)

Если IP-адрес клиента присутствует в конфигурационном белом списке доверенных адресов, запрос немедленно проксируется к бэкенду без дальнейшего анализа. Это необходимо для исключения блокировки администраторов или доверенных систем мониторинга.

4. Проверка состояния блокировки

Система проверяет, не заблокирован ли данный IP-адрес ранее. Проверка осуществляется в два этапа: сначала в оперативной памяти (для

быстрого доступа), затем в базе данных SQLite (таблица BlockedIP). Если IP уже заблокирован, запрос немедленно отклоняется с кодом 403 Forbidden без выполнения каких-либо дополнительных проверок.

5. Трекинг активности в скользящем окне

Для каждого IP-адреса ведётся учёт активности в скользящем временном окне продолжительностью 60 секунд. В рамках этого окна фиксируются: все запрашиваемые URL, количество запросов с кодом ответа 404 (не найдено), общее количество запросов. Окно постоянно обновляется – устаревшие записи удаляются.

6. Проверка трёх пороговых детекторов

Система последовательно проверяет три пороговых значения, каждое из которых свидетельствует об определённом типе атаки:

Детектор №1 – сканирование директорий: если количество уникальных URL, запрошенных с одного IP за последние 60 секунд, превышает или равно 50, это указывает на попытку перебора директорий и файлов (dirbusting).

Детектор №2 – сканирование уязвимостей (404-детектор): если доля запросов, завершившихся с кодом 404, составляет 70% и более от общего числа запросов (при условии, что общее количество запросов не менее 10), это свидетельствует о сканировании несуществующих путей в поисках уязвимостей.

Детектор №3 – DoS/DDoS-атака: если количество запросов в секунду (RPS) от одного IP превышает или равно 30, это указывает на попытку перегрузки сервера (отказ в обслуживании).

Если срабатывает хотя бы один из трёх детекторов, ситуация классифицируется как атака. В противном случае трафик считается легитимным.

7А. Блокировка при обнаружении атаки

При срабатывании любого детектора IP-адрес заносится в таблицу BlockedIP базы данных с указанием причины и временной метки. Запись также кэшируется в памяти для ускорения последующих проверок. Клиенту

возвращается HTTP-ответ с кодом 403 Forbidden. Все последующие запросы с этого IP будут отклоняться на шаге 4 без дополнительного анализа.

7Б. Проксирование при нормальном трафике

Если ни один из детекторов не сработал, запрос считается безопасным и проксируется к защищаемому бэкенд-приложению. URL бэкенда задаётся в конфигурационном файле `waf_config.yaml`. При отсутствии настроенного бэкенда запрос обрабатывается непосредственно WAF-приложением.

8. Логирование всех событий

Независимо от результата проверки (блокировка или проксирование), информация о запросе сохраняется в трёх местах:

Таблица `SecurityEvent` SQLite – структурированное хранилище событий безопасности (время, IP, метод, URL, сработавший детектор, действие);

Таблица `RequestLog` SQLite – архив всех обработанных запросов (для аудита и аналитики);

JSON-файл `logs/security_events.json` – структурированные логи в формате, совместимом с ELK-стеком (Elasticsearch, Logstash, Kibana) для централизованного сбора и анализа.

9. Формирование ответа клиенту

В зависимости от принятого решения клиенту возвращается либо ответ 403 Forbidden (при блокировке), либо ответ, полученный от проксированного бэкенд-приложения (при успешном прохождении проверок).

Итоговая схема принятия решений

Алгоритм работает по принципу "отказ по умолчанию" – только запросы, успешно прошедшие все проверки (не системный путь, не в `whitelist`, не заблокирован, не превысил ни один порог), проксируются к бэкенду. Любое отклонение от нормы приводит к блокировке или дополнительной проверке. Такой подход обеспечивает многоуровневую защиту от различных типов атак: сканирования директорий, поиска уязвимостей и DoS/DDoS-атак.

Блок схема работы контейнера представлена на рисунке 3.

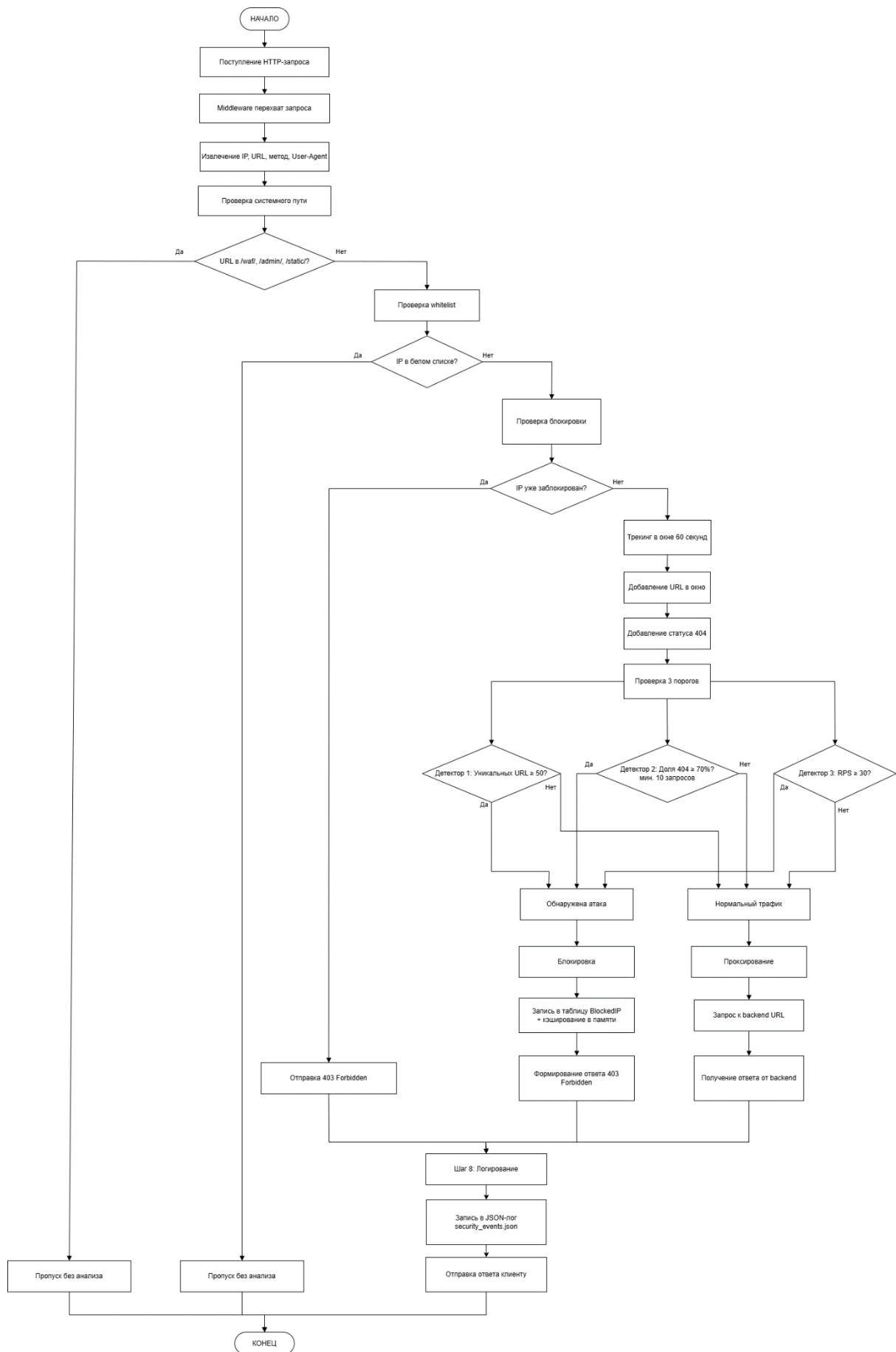


Рисунок 3 - Блок-схема к контейнеру безопасности

Вывод

В ходе лабораторной работы я приобрела навыки конструирования систем с элементами ИИ на базе паттерна Python Data Science в SaaS Replit. А также реализовала контейнер по обеспечению безопасности при угрозе форсированного веб-браузинга, и подготовила полный пакет программной документации.

УТВЕРЖДАЮ
Начальник тех.отдела
_____Корнеев Н.В.
“29” мая 2026

**ПРОГРАММА ОБЕСПЕЧЕНИЯ ЗАЩИТЫ ОТ АТАКИ
«ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА» (УБИ.159)**

Спецификация

Лист утверждения

XXXXXXXX.XXXXXXX.001

Подпись и	
Инв. №	
Взам.	
Подпись и	
Инв. №	

Руководитель разработки
Корнеев Н.В.
“29” мая 2026

Ответственный исполнитель

Вотинцева А.С.
“29” мая 2026

УТВЕРЖДЕН

XXXXXXX.XXXXXXX.001

ПРОГРАММА ОЧИСТКИ ОПЕРАТИВНОЙ ПАМЯТИ

Спецификация

XXXXXXX.XXXXXXX.001

Инд. №	Подпись и	Взам.	Инд.	Подпись и

Настоящая спецификация распространяется на программное изделие «WAF-контейнер» и устанавливает состав программного изделия и документации на него. Спецификация составлена в соответствии с требованиями ГОСТ 19.202-78 «Единая система программной документации. Спецификация. Требования к содержанию и оформлению».

Таблица 3 - Документация

Обозначение	Наименование	Примечание
XXXXXX.XXXXXX.001	Спецификация	Настоящий документ
XXXXXX.XXXXXX.002	Техническое задание	WAF-контейнер
XXXXXX.XXXXXX.003	Текст программы	Исходный код WAF-контейнера
XXXXXX.XXXXXX.004	Описание программы	WAF-контейнер
XXXXXX.XXXXXX.005	Пояснительная записка	WAF-контейнер
XXXXXX.XXXXXX.006	Программа и методика испытаний	WAF-контейнер
XXXXXX.XXXXXX.ЭД1	Ведомость эксплуатационных документов	—
XXXXXX.XXXXXX.ЭД2	Формуляр	—
XXXXXX.XXXXXX.ЭД3	Руководство оператора	—
XXXXXX.XXXXXX.ЭД4	Руководство программиста	—

Комплексы в состав программного изделия не входят.

Таблица 4 - Компоненты

Обозначение	Наименование	Описание
waf.middleware	ForcedBrowsingProtectionMiddleware	Перехват HTTP-запросов, проксирование, вызов движка защиты
waf.protection	ProtectionEngine (Singleton)	Скользящее окно, проверка порогов, блокировка IP
waf.protection	IPTracker	Трекер активности одного IP: deque уникальных URL и 404-ошибок
waf.models	BlockedIP	Модель заблокированных IP-адресов (SQLite)
waf.models	SecurityEvent	Журнал событий безопасности (аудит-трейл)
waf.models	RequestLog	Полный лог HTTP-запросов для графиков дашборда
waf.models	TrustedIP	Белый список доверенных IP-адресов и подсетей
waf.views	dashboard / api_*	HTML-дашборд и 6 REST API-эндпоинтов управления
waf.logger	JsonFormatter / log_event	ELK-совместимый JSON-форматтер событий безопасности
config/waf_config.yaml	Конфигурационный файл	Пороги, режим работы, whitelist, backend URL

УТВЕРЖДАЮ

Начальник тех.отдела

_____Корнеев Н.В.

“29” мая 2026

**ПРОГРАММА ОБЕСПЕЧЕНИЯ ЗАЩИТЫ ОТ АТАКИ
«ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА» (УБИ.159)**

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Лист утверждения

XXXXXXXX.XXXXXXX.002

Подпись и	
Инв. №	
Взам.	
Подпись и	
Инв. №	

Руководитель разработки

Корнеев Н.В.

“29” мая 2026

Ответственный исполнитель

Вотинцева А.С.

“29” мая 2026

УТВЕРЖДЕНО
XXXXXXX.XXXXXXX.002

ПРОГРАММА ОЧИСТКИ ОПЕРАТИВНОЙ ПАМЯТИ

ТЕХНИЧЕСКОЕ ЗАДАНИЕ
XXXXXXX.XXXXXXX.002

Инов. №	Подпись и	Инов.	Взам.	Инов.	Подпись и

1 Введение

Настоящее техническое задание распространяется на разработку программного изделия «WAF-контейнер защиты от атаки форсированного веб-браузинга» (далее — WAF-контейнер). Программное изделие предназначено для обнаружения и блокирования атак класса «Форсированный веб-браузинг» (УБИ.159 по методике ФСТЭК России) на веб-приложения.

Область применения: информационные системы, требующие защиты веб-интерфейсов от автоматизированного перебора URL-адресов и сканирования скрытых ресурсов.

2 Основания для разработки

Основанием для разработки является учебное задание по дисциплине «Управление жизненным циклом ИС». Тема разработки: «WAF-контейнер защиты от атаки форсированного веб-браузинга (УБИ.159)».

3 Назначение разработки

Функциональное назначение: обнаружение и блокирование атак форсированного веб-браузинга путём анализа входящих HTTP-запросов с применением метода скользящего временного окна.

Эксплуатационное назначение: развёртывание в качестве обратного прокси-сервера перед защищаемым веб-приложением с минимальной конфигурацией через YAML-файл.

4 Требования к программе

4.1 Требования к функциональным характеристикам

WAF-контейнер должен выполнять следующие функции:

- перехват всех входящих HTTP-запросов до передачи защищаемому приложению;
- подсчёт уникальных URL per-IP в скользящем временном окне (по умолчанию 60 с);
- подсчёт доли ответов с кодом 404 (HTTP Not Found) в скользящем окне;
- вычисление частоты запросов (RPS — requests per second) на IP;
- автоматическая блокировка IP при превышении любого из трёх пороговых значений;

- проксирование разрешённых запросов на защищаемый backend-сервер;
- ведение журнала событий безопасности в формате JSON (ELK-совместимый);
- HTML-дашборд с Chart.js-визуализацией статистики и управлением блокировками;
- три режима работы: block (блокировка), monitoring (мониторинг), disabled (отключён);
- REST API для управления блокировками и получения статистики;
- поддержка белого списка IP-адресов и подсетей (CIDR-нотация);
- горячая перезагрузка конфигурации без перезапуска сервиса.

4.2 Требования к надёжности

1. Отказ подсистемы логирования не должен прерывать обработку запросов.
2. Отказ backend-сервера должен возвращать код 502 Bad Gateway без краша WAF.
3. Счётчики защищены мьютексами (threading.Lock) для потокобезопасности.
4. Фоновый поток очистки устаревших данных завершается вместе с процессом (daemon=True).
5. Конфиг читается с fallback на безопасные значения по умолчанию при ошибке парсинга.

Таблица 5 - 4.3 Условия эксплуатации

Параметр	Требование
Операционная система	Linux (рекомендуется), Windows, macOS
Сеть	TCP/IP, порт 8000 (настраиваемый)
Количество персонала	Минимум 1 администратор
Квалификация	Знание основ HTTP, опыт работы с Linux CLI
Режим работы	Непрерывный (24/7)

Таблица 6 - 4.4 Требования к составу и параметрам технических средств

Компонент	Минимальные требования
ЦП	x86-64, 1 ядро, 1 ГГц
ОЗУ	256 МБ (рекомендуется 512 МБ)
Диск	500 МБ свободного места
Сеть	100 Мбит/с Ethernet
ПО	Python 3.11+, pip

4.5 Требования к информационной и программной совместимости

- Язык программирования: Python 3.11.
- Веб-фреймворк: Django 4.2.x.
- СУБД: SQLite 3 (встроена в Python); возможна замена на PostgreSQL.
- Формат конфигурации: YAML (PyYAML ≥ 6.0).

Входные данные: HTTP/1.1 и HTTP/2 запросы.

Выходные данные: JSON-события безопасности (NDJSON, ELK-совместимые).

HTTP-клиент для проксирования: requests ≥ 2.31 .

Статические файлы: WhiteNoise ≥ 6.0 .

5 Требования к программной документации

В состав программной документации должны входить:

1. Спецификация (ГОСТ 19.202-78)
2. Техническое задание (ГОСТ 19.201-78)
3. Текст программы (ГОСТ 19.401-78)
4. Описание программы (ГОСТ 19.402-78)
5. Пояснительная записка (ГОСТ 19.404-79)
6. Программа и методика испытаний (ГОСТ 19.301-79)
7. Руководство оператора (ГОСТ 19.505-79)
8. Руководство программиста (ГОСТ 19.504-79)
9. Формуляр (ГОСТ 19.501-78)

6 Техничко-экономические показатели

Экономическая эффективность обеспечивается снижением ущерба от атак форсированного веб-браузинга: предотвращение утечки конфиденциальных файлов, исключение несанкционированного доступа к

административным интерфейсам. Ориентировочный эффект — снижение успешных атак класса УБИ.159 на $\geq 95\%$.

Таблица 7 - 7 Стадии и этапы разработки

Стадия	Этап	Содержание работ
Техническое задание	Анализ угроз	Изучение УБИ.159, Wallarm Forced Browsing Protection
Эскизный проект	Проектирование	Архитектура middleware, алгоритм скользящего окна
Технический проект	Детальное проектирование	Схемы модулей, модели данных, API
Рабочий проект	Кодирование	Реализация всех компонентов, тесты
Внедрение	Испытания	Функциональное тестирование, нагрузочные тесты

8 Порядок контроля и приёмки

Приёмка выполняется путём проведения испытаний согласно «Программе и методике испытаний» (XXXXXX.XXXXXX.006). Виды испытаний: функциональное тестирование, симуляция атаки, проверка режимов работы, проверка API.

УТВЕРЖДАЮ
Начальник тех.отдела
_____Корнеев Н.В.
“29” мая 2026

**ПРОГРАММА ОБЕСПЕЧЕНИЯ ЗАЩИТЫ ОТ АТАКИ
«ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА» (УБИ.159)**

ТЕКСТ ПРОГРАММЫ

Лист утверждения

XXXXXXXX.XXXXXXXXX.003

Подпись и	
Инв. №	
Взам.	
Подпись и	
Инв. №	

Руководитель разработки
Корнеев Н.В.
“29” мая 2026

Ответственный исполнитель

Вотинцева А.С.
“29” мая 2026

УТВЕРЖДЕНО
XXXXXXX.XXXXXXX.003

ПРОГРАММА ОЧИСТКИ ОПЕРАТИВНОЙ ПАМЯТИ

ТЕКСТ ПРОГРАММЫ
XXXXXXX.XXXXXXX.003

Инд. №	Подпись и	Взам.	Инд.	Подпись и

Настоящий документ содержит текст программы WAF-контейнера.
Составлен в соответствии с ГОСТ 19.401-78.

Модуль 1: waf/protection.py — Движок защиты (ProtectionEngine)

```
# Класс IPTracker — трекер активности одного IP
class IPTracker:
    def __init__(self, window_seconds: int):
        self.window_seconds = window_seconds
        self.lock = threading.Lock()
        self.unique_urls: set = set()
        self.error_404_times: deque = deque()
        self.all_request_times: deque = deque()

    def add_request(self, url, status_code, ts):
        with self.lock:
            cutoff = ts - self.window_seconds
            # Удаление устаревших записей (скользящее окно)
            while self.all_request_times and \
                self.all_request_times[0] < cutoff:
                self.all_request_times.popleft()
            self.all_request_times.append(ts)
            if status_code == 404:
                self.error_404_times.append(ts)
            self.unique_urls.add(url)

# Класс ProtectionEngine — Singleton движка защиты
class ProtectionEngine:
    _instance = None
    _lock = threading.Lock()

    @classmethod
    def get_instance(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = cls()
        return cls._instance

    def check_and_record(self, ip, url, path, method,
                        status_code, user_agent=""):
        """Главный метод: анализирует запрос, возвращает
        решение."""
        result = {"action": "allow", "reason": None, "stats":
        {}}

        if self._is_whitelisted(ip):
            result["action"] = "whitelist"
```

```

        return result
    if self.is_blocked(ip):
        result["action"] = "block"
        return result
    tracker = self._get_tracker(ip)
    tracker.add_request(url, status_code, time.time())
    stats = tracker.get_stats(time.time())
    # Проверка трёх порогов обнаружения атаки
    thresholds = self.config.get("thresholds", {})
    if stats["unique_urls"] >=
thresholds.get("unique_urls", 50):
        violation_reason = "unique_url_limit"
    elif stats["error_rate"] >=
thresholds.get("error_rate_percent", 70):
        violation_reason = "error_rate_limit"
    elif rps >= thresholds.get("rps", 30):
        violation_reason = "rps_limit"
    if violation_reason and mode == "block":
        self._block_ip(ip, violation_reason, stats)
        result["action"] = "block"
    return result

```

Модуль 2: waf/middleware.py — Middleware перехвата запросов

```

class ForcedBrowsingProtectionMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        from waf.protection import ProtectionEngine
        engine = ProtectionEngine.get_instance()
        ip = get_client_ip(request)
        # Проверка исключённых путей (/waf/, /admin/,
/static/)
        if any(request.path.startswith(e)
                for e in engine.config.get("excluded_paths",
[]))):
            return self.get_response(request)
        # Предварительная проверка блокировки
        if engine.is_blocked(ip):
            return self._blocked_response(engine)
        # Демо-режим: Django сам обрабатывает запрос
        if not backend_url:
            response = self.get_response(request)
            engine.check_and_record(ip, url, path,
                                   method, response.status_code, user_agent)

```

```

        return response
    # Production-режим: проксирование на backend
    proxy_resp = self._proxy_request(request,
backend_url)
    check_result = engine.check_and_record(...)
    if check_result["action"] == "block":
        return self._blocked_response(engine)
    return self._build_django_response(proxy_resp)

```

Модуль 3: waf/models.py — Модели базы данных

```

class BlockedIP(models.Model):
    ip_address = models.GenericIPAddressField(unique=True)
    blocked_at = models.DateTimeField(default=timezone.now)
    expires_at = models.DateTimeField(null=True, blank=True)
    reason = models.CharField(max_length=50,
choices=BLOCK_REASON_CHOICES)
    is_active = models.BooleanField(default=True)

class SecurityEvent(models.Model):
    timestamp = models.DateTimeField(default=timezone.now)
    event_type = models.CharField(max_length=30,
choices=EVENT_TYPE_CHOICES)
    source_ip = models.GenericIPAddressField()
    url = models.TextField()
    unique_urls_at_time = models.IntegerField(default=0)
    error_rate = models.FloatField(default=0.0)
    details = models.JSONField(default=dict)

class RequestLog(models.Model):
    source_ip = models.GenericIPAddressField()
    url = models.TextField()
    status_code = models.IntegerField(null=True)
    was_blocked = models.BooleanField(default=False)
    response_time_ms = models.FloatField(null=True)

```

Модуль 4: waf/logger.py — ELK-совместимое логирование

```

class JsonFormatter(logging.Formatter):
    def format(self, record):
        log_entry = {
            "timestamp":
datetime.now(timezone.utc).isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
        }
        if hasattr(record, "extra"):

```

```
        log_entry.update(record.extra)
    return json.dumps(log_entry, ensure_ascii=False)

def log_event(event_type, ip, url="", **kwargs):
    extra = {"event_type": event_type, "source_ip": ip,
            "url": url, **kwargs}
    security_logger.info(f"{event_type}: {ip} -> {url}",
                        extra={"extra": extra})
```

УТВЕРЖДАЮ
Начальник тех.отдела

_____Корнеев Н.В.

“29” мая 2026

**ПРОГРАММА ОБЕСПЕЧЕНИЯ ЗАЩИТЫ ОТ АТАКИ
«ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА» (УБИ.159)**

ОПИСАНИЕ ПРОГРАММЫ

Лист утверждения

XXXXXXXX.XXXXXXX.004

Подпись и	
Инв. №	
Взам.	
Подпись и	
Инв. №	

Руководитель разработки

Корнеев Н.В.

“29” мая 2026

Ответственный исполнитель

Вотинцева А.С.

“29” мая 2026

УТВЕРЖДЕНО
XXXXXXX.XXXXXXX.004

ПРОГРАММА ОЧИСТКИ ОПЕРАТИВНОЙ ПАМЯТИ

ОПИСАНИЕ ПРОГРАММЫ
XXXXXXX.XXXXXXX.004

Инд. №	Подпись и	Взам.	Инд.	Подпись и

1 Общие сведения о программе представлены в таблице 8.

Таблица 8 - Общие сведения

Параметр	Значение
Обозначение	XXXXXX.XXXXXX
Наименование	WAF-контейнер защиты от атаки форсированного веб-браузинга
Аббревиатура	WAF (Web Application Firewall)
Язык программирования	Python 3.11
Фреймворк	Django 4.2
СУБД	SQLite 3
Конфигурация	YAML (waf_config.yaml)
Платформа	Linux / Windows / macOS
Порт по умолчанию	8000

2 Функциональное назначение

WAF-контейнер предназначен для защиты веб-приложений от атак класса «Форсированный веб-браузинг» (УБИ.159 по методике ФСТЭК России). Программа реализует аналог функциональности модуля Wallarm Forced Browsing Protection.

Классы решаемых задач:

- Обнаружение автоматизированного перебора URL-адресов (словарные атаки).
- Блокирование IP-адресов, превысивших допустимые пороги активности.
- Мониторинг и визуализация трафика в реальном времени.
- Аудит событий безопасности в ELK-совместимом формате.

Функциональные ограничения:

Блокировка выполняется на уровне IP (не на уровне сессий/пользователей).

В текущей версии используется SQLite; для высоконагруженных систем рекомендуется Redis.

3 Описание логической структуры

Алгоритм работы программы:

- (1) Перехват запроса в Django Middleware.
- (2) Извлечение IP-адреса (с учётом X-Forwarded-For).

- (3) Фильтрация исключённых системных путей.
 - (4) Проверка белого списка (whitelist).
 - (5) Проверка наличия активной блокировки IP (БД + in-memory).
 - (6) Обновление IPTracker: добавление URL и timestamp в скользящее окно.
 - (7) Вычисление метрик: unique_urls, error_rate, RPS.
 - (8) Сравнение метрик с пороговыми значениями из waf_config.yaml.
 - (9) При нарушении порога: блокировка IP, запись события в БД и JSON-лог.
 - (10) Проксирование разрешённых запросов на backend или обработка Django.
 - (11) Возврат клиенту: 403 Forbidden (блокировка) или ответ backend.
- Используемые методы:
- Скользящее временное окно (Sliding Window) на основе deque — $O(1)$ добавление/удаление.
 - Паттерн Singleton для ProtectionEngine — единое состояние счётчиков в процессе.
 - Double-checked locking — потокобезопасное создание Singleton без избыточной блокировки.
 - Множество (set) для подсчёта уникальных URL — автоматическая дедупликация за $O(1)$.

Таблица 9 - Структура модулей и связи

Модуль	Зависит от	Описание связи
middleware.py	protection.py	Вызов check_and_record() для каждого запроса
middleware.py	models.py	Запись RequestLog через _save_request_log()
protection.py	models.py	Запись BlockedIP и SecurityEvent
protection.py	logger.py	Вызов log_event() при блокировке/атаке
views.py	protection.py	Получение статистики, ручное управление блокировками
views.py	models.py	Агрегация данных для дашборда
settings.py	middleware.py	Регистрация в MIDDLEWARE[]
waf_config.yaml	protection.py	Загрузка при инициализации ProtectionEngine

4 Используемые технические средства

Программа функционирует на ПЭВМ архитектуры x86-64 под управлением операционной системы Linux (рекомендуется Ubuntu 22.04 LTS / Debian 12). Дополнительного специализированного оборудования не требуется.

5 Вызов и загрузка

Запуск сервера разработки:

```
python manage.py runserver 0.0.0.0:8000
```

Инициализация БД (первый запуск):

```
python manage.py migrate
python manage.py seed_demo # загрузка демо-данных
```

ProtectionEngine инициализируется автоматически при обработке первого запроса через метод `get_instance()` (Singleton). Конфигурация загружается из `config/waf_config.yaml`.

6 Входные данные представлены в таблице 10.

Таблица 10 - Входные данные

Входной поток	Формат	Описание
HTTP-запрос	HTTP/1.1, HTTP/2	Входящий запрос от клиента: метод, URL, заголовки
waf_config.yaml	YAML	Конфигурация: пороги, режим, whitelist, backend URL
X-Forwarded-For	HTTP-заголовок	Реальный IP клиента за обратным прокси
Тело запроса	произвольный	Для POST/PUT/PATCH — передаётся на backend as-is

7 Выходные данные представлены в таблице 11.

Таблица 11 - Выходные данные

Выходной поток	Формат	Описание
HTTP-ответ клиенту	HTTP/1.1	200 OK (разрешён) / 403 Forbidden (заблокирован)
security_events.json	NDJSON	Структурированные события безопасности (ELK)
SQLite BlockedIP	SQL	Заблокированные IP: адрес, причина, срок
SQLite SecurityEvent	SQL	Журнал событий: тип, IP, URL, метрики
SQLite RequestLog	SQL	Лог запросов: метод, URL, статус, время
REST API /waf/api/stats/	JSON	Агрегированная статистика для дашборда
HTML дашборд /waf/	HTML+JS	Визуализация: Chart.js графики, таблицы

Приложение А
УТВЕРЖДАЮ
Начальник тех.отдела
_____Корнеев Н.В.
“29” мая 2026

**ПРОГРАММА ОБЕСПЕЧЕНИЯ ЗАЩИТЫ ОТ АТАКИ
«ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА» (УБИ.159)**

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

Лист утверждения

XXXXXXXX.XXXXXXX.005

Подпись и	
Инв. №	
Взам.	
Подпись и	
Инв. №	

Руководитель разработки
Корнеев Н.В.
“29” мая 2026

Ответственный исполнитель

Вотинцева А.С.
“29” мая 2026

УТВЕРЖДЕНО
XXXXXXX.XXXXXXX.005

ПРОГРАММА ОЧИСТКИ ОПЕРАТИВНОЙ ПАМЯТИ

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
XXXXXXX.XXXXXXX.005

Инд. №	Подпись и
Взам.	Инд.
Подпись и	

1 Введение

Настоящая пояснительная записка относится к программному изделию «WAF-контейнер защиты от атаки форсированного веб-браузинга» и составлена на стадии технического проекта согласно ГОСТ 19.404-79.

Атака «Форсированный веб-браузинг» (УБИ.159 по банку угроз ФСТЭК России) заключается в автоматизированном переборе URL-адресов с целью получения несанкционированного доступа к скрытым ресурсам веб-приложения: административным панелям, конфигурационным файлам, резервным копиям, API-эндпоинтам без аутентификации.

2 Назначение и область применения

WAF-контейнер предназначен для встраивания в инфраструктуру веб-приложений в качестве обратного прокси-сервера. Область применения: корпоративные информационные системы, интернет-порталы, сервисы электронного правительства.

3 Технические характеристики

3.1 Постановка задачи

Задача — разработать программный модуль, реализующий следующий алгоритм обнаружения атаки форсированного браузинга:

Для каждого IP-адреса поддерживается скользящее временное окно размером W секунд. В окне отслеживаются: множество уникальных URL $U(ip,t)$, последовательности временных меток запросов $T(ip,t)$ и ошибок 404 $E(ip,t)$. IP блокируется если выполняется хотя бы одно из условий:

$|U(ip,t)| \geq \text{unique_url_threshold}$ (по умолчанию 50)

$|E(ip,t)| / |T(ip,t)| \times 100\% \geq \text{error_rate_threshold}$ (по умолчанию 70%)

$|T(ip,t)| / W \geq \text{rps_threshold}$ (по умолчанию 30 запросов/сек)

3.2 Описание алгоритма и функционирования

Алгоритм построен на структуре данных «скользящее окно» (Sliding Window). Реализация использует `collections.deque` — двустороннюю очередь с $O(1)$ операциями добавления в конец и удаления с начала. Все операции со

счётчиками защищены `threading.Lock` для потокобезопасности в многопоточном WSGI-окружении Django.

ProtectionEngine реализован по паттерну Singleton с double-checked locking, что обеспечивает единственный экземпляр и общую память счётчиков между воркерами одного процесса Django. В production-среде с несколькими процессами рекомендуется замена in-memory счётчиков на Redis.

3.3 Организация входных и выходных данных

Входные данные: поток HTTP-запросов, принимаемых Django-сервером. Каждый запрос обрабатывается синхронно в цепочке middleware. Конфигурация загружается из YAML-файла при старте движка.

Выходные данные: (1) HTTP-ответы клиентам (403/200/проксированный ответ); (2) записи в SQLite-таблицах (BlockedIP, SecurityEvent, RequestLog); (3) JSON-события в файле logs/security_events.json; (4) данные REST API для дашборда.

3.4 Обоснование выбора технических и программных средств

Таблица 12 - Обоснование выбора программных средств

Компонент	Выбор	Обоснование
Язык	Python 3.11	Богатая экосистема, встроенные threading, collections
Фреймворк	Django 4.2	Встроенные middleware, ORM, admin; зрелость
СУБД	SQLite	Достаточна для лаборатории; нет внешних зависимостей
Конфиг	YAML	Читаемость, поддержка вложенных структур, комментарии
Статика	WhiteNoise	Раздача static без nginx в dev/prod
Визуализация	Chart.js	Клиентская библиотека без серверного рендеринга
Логи	NDJSON	ELK-совместимость: прямой импорт в Logstash/Filebeat

4 Ожидаемые технико-экономические показатели

Таблица 13 - Техничко-экономические результаты

Показатель	Значение
Задержка на запрос (overhead)	< 1 мс (in-memory операции)
Пропускная способность	> 1000 RPS на одном ядре ЦП
Память на 10 000 активных IP	≈ 50 МБ (deque + set)
Время до блокировки	≤ 60 секунд (размер окна)
Точность обнаружения	≥ 95% (при настройках по умолчанию)
Ложные блокировки	< 0.1% легитимного трафика

5 Источники, использованные при разработке

Нормативно-технические документы:

1. ГОСТ 19.101-2024 "Единая система программной документации. Виды программ и программных документов".
2. ГОСТ 19.102-77 "Единая система программной документации. Стадии разработки".
3. ГОСТ 19.103-77 "Единая система программной документации. Обозначение программ и программных документов".
4. ГОСТ 19.104-78 "Единая система программной документации. Основные надписи".
5. ГОСТ 19.105-78 "Единая система программной документации. Общие требования к программным документам".
6. ГОСТ 19.106-78 "Единая система программной документации. Требования к программным документам, выполненным печатным способом".
7. ГОСТ 19.201-78 "Единая система программной документации. Техническое задание. Требования к содержанию и оформлению".
8. ГОСТ 19.202-78 "Единая система программной документации. Спецификация. Требования к содержанию и оформлению".
9. ГОСТ 19.301-79 "Единая система программной документации. Программа и методика испытаний. Требования к содержанию и оформлению".
10. ГОСТ 19.401-78 "Единая система программной документации. Текст программы. Требования к содержанию и оформлению".
11. ГОСТ 19.402-78 "Единая система программной документации. Описание программы. Требования к содержанию и оформлению".
12. ГОСТ 19.404-79 "Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению".

Приложение А
УТВЕРЖДАЮ
Начальник тех.отдела
_____Корнеев Н.В.
“29” мая 2026

**ПРОГРАММА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПРИ
УГРОЗЕ ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА**

Программа и методика испытаний

Лист утверждения

XXXXXX.XXXXXX.006

Подпись и	
Инв. №	
Взам.	
Подпись и	
Инв. №	

Руководитель разработки
Корнеев Н.В.
“29” мая 2026

Ответственный исполнитель
Вотинцева А.С.
“29” мая 2026

УТВЕРЖДЕНО

XXXXXXX.XXXXXXX.006

**ПРОГРАММА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПРИ
УГРОЗЕ ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА**

Программа и методика испытаний

XXXXXXX.XXXXXXX.006

Инв. №	Подпись и	Взам.	Инв.	Подпись и

1. Объект испытаний

Объект испытаний: программное изделие «WAF-контейнер защиты от атаки форсированного веб-браузинга», обозначение XXXXXXXX.XXXXXXX.
Область применения: защита веб-приложений от атак класса УБИ.159.

2. Цель испытаний

Цель испытаний: проверка соответствия WAF-контейнера требованиям, изложенным в техническом задании (XXXXXXX.XXXXXXX.002), по функциональности, надёжности и корректности обнаружения атак.

3. Требования к программе

- Блокировка IP при превышении порога уникальных URL (≥ 50).
- Блокировка IP при превышении доли 404-ошибок ($\geq 70\%$, мин. 10 запросов).
- Блокировка IP при превышении RPS (≥ 30 запросов/сек).
- В режиме monitoring: события фиксируются, блокировка не выполняется.
- В режиме disabled: WAF прозрачен, все запросы пропускаются.
- IP из whitelist не блокируется никогда.
- Корректная запись событий в SQLite и JSON-лог.
- REST API возвращает корректные данные статистики.
- HTML-дашборд отображается без ошибок на /waf/.

4. Требования к программной документации

На испытания предъявляется полный комплект документации ЕСПД: Спецификация, ТЗ, Текст программы, Описание программы, Пояснительная записка, Руководство оператора, Руководство программиста.

5. Состав и порядок испытаний

Таблица 14 - Состав и порядок испытаний

№	Тест	Входные данные	Ожидаемый результат
T-01	Блокировка по unique_urls	55 запросов к уникальным URL от одного IP	IP блокируется, HTTP 403, запись в BlockedIP
T-02	Блокировка по error_rate	15 запросов, 12 из них с ответом 404 (80%)	IP блокируется, HTTP 403
T-03	Блокировка по RPS	35 запросов в течение 1 секунды	IP блокируется, HTTP 403
T-04	Whitelist	Запросы от 127.0.0.1 (в whitelist)	IP не блокируется, все запросы пропускаются
T-05	Режим monitoring	mode: monitoring, 55 уникальных URL	Событие записано, блокировки нет, HTTP 200
T-06	Режим disabled	mode: disabled, любые запросы	WAF прозрачен, все запросы проходят
T-07	Ручная блокировка	POST /waf/api/block-ip/ {"ip": "1.2.3.4"}	IP блокируется, HTTP 403 при следующем запросе
T-08	Ручная разблокировка	POST /waf/api/unblock-ip/ {"ip": "1.2.3.4"}	IP разблокирован, запросы проходят
T-09	Статистика API	GET /waf/api/stats/	JSON с корректными счётчиками
T-10	Симулятор атаки	POST /waf/api/simulate/ {"count": 70}	Атака обнаружена, IP заблокирован
T-11	HTML дашборд	GET /waf/	HTTP 200, страница отображается с Chart.js
T-12	Очистка логов	POST /waf/api/clear-logs/	Все таблицы очищены, трекеры сброшены

6. Методы испытаний

Испытания проводятся методом «чёрного ящика» (функциональное тестирование). Инструменты: curl, HTTP-клиент браузера, встроенный симулятор атаки на дашборде /waf/.

Методика проведения теста T-01:

1. Запустить WAF-сервер: `python manage.py runserver 0.0.0.0:8000`
2. Убедиться что mode: block в waf_config.yaml
3. Выполнить POST /waf/api/simulate/ с телом {"count": 60}
4. Проверить GET /waf/api/stats/ — поле active_blocks должно быть ≥ 1
5. Выполнить GET /waf/api/events/?type=attack_detected — событие должно присутствовать
6. Выполнить запрос от заблокированного IP — ожидать HTTP 403

Критерий успешного прохождения испытаний: все тесты Т-01 — Т-12 дают ожидаемые результаты. При отклонении результата от ожидаемого — тест считается проваленным, требуется анализ и устранение дефекта.

Порядок проведения испытаний

1. Подготовка к испытаниям:

- установить Python версии 3.8 или выше;
- установить Django версии 4.2 или выше;
- загрузить исходный код программы;
- запустить Django-сервер командой: `python manage.py runserver`.

2. Проведение испытаний:

- открыть веб-браузер;
- перейти по адресу: `http://localhost:8000`;
- дождаться загрузки страницы с результатами;
- зафиксировать полученные результаты.

3. Обработка результатов:

- сравнить полученные результаты с ожидаемыми;
- заполнить протокол испытаний;
- оформить заключение о соответствии программы требованиям.

Метод проверки функциональных характеристик

Таблица 14 - Тест-кейс 1. Проверка выявления административных панелей

Параметр	Значение
Входные данные	GET /admin/, GET /wp-admin/, GET /administrator/
Ожидаемый результат	allowed = False, category = "Адм. панели", status_code = 403
Метод проверки	Визуальный контроль в таблице результатов

Таблица 15 - Тест-кейс 2. Проверка выявления системных файлов

Параметр	Значение
Входные данные	GET /.git/config, GET /.env, GET /config/
Ожидаемый результат	allowed = False, category = "Системные файлы", status_code = 403
Метод проверки	Визуальный контроль в таблице результатов

Таблица 16 - Тест-кейс 3. Проверка выявления бэкапов

Параметр	Значение
Входные данные	GET /backup/, GET /back.sql, GET /tmp/

Ожидаемый результат	allowed = False, category = "Бэкапы/Временные", status_code = 403
Метод проверки	Визуальный контроль в таблице результатов

Таблица 17 -Тест-кейс 4. Проверка выявления чувствительных данных

Параметр	Значение
Входные данные	GET /private/, GET /passwd, GET /secret/
Ожидаемый результат	allowed = False, category = "Чувств. данные", status_code = 403
Метод проверки	Визуальный контроль в таблице результатов

Таблица 18 - Тест-кейс 5. Проверка выявления скрытых файлов

Параметр	Значение
Входные данные	GET /.htaccess, GET /.svn/, GET /.idea/
Ожидаемый результат	allowed = False, category = "Скрытые файлы", status_code = 403
Метод проверки	Визуальный контроль в таблице результатов

Таблица 19 - Тест-кейс 6. Проверка разрешённых путей

Параметр	Значение
Входные данные	GET /, GET /about/, GET /contacts/
Ожидаемый результат	allowed = True, category = "Разрешённый", status_code = 200
Метод проверки	Визуальный контроль в таблице результатов

Таблица 20 - Тест-кейс 7. Проверка несуществующих путей

Параметр	Значение
Входные данные	GET /nonexistent123, GET /old-page/
Ожидаемый результат	allowed = False, category = "Не найден", status_code = 404
Метод проверки	Визуальный контроль в таблице результатов

Таблица 21 - Тест-кейс 8. Проверка общей статистики

Параметр	Ожидаемое значение
total	32
blocked	27
allowed	5
block_rate	84.4%

Таблица 22 - Тест-кейс 9. Проверка круговой диаграммы

Параметр	Ожидаемое значение
Сектор "Заблокировано"	84.4%
Сектор "Разрешено"	15.6%

Таблица 23 - Тест-кейс 10. Проверка столбчатой диаграммы

Категория	Ожидаемое количество
Адм. панели	5
Системные файлы	5
Бэкапы/Временные	5

Чувств. данные	4
Скрытые файлы	4
Разрешённый	5
Не найден	2

Таблица 24 - Результаты испытаний

№ тест-кейса	Наименование проверки	Результат	Примечание
1	Административные панели	_____	
2	Системные файлы	_____	
3	Бэкапы и временные файлы	_____	
4	Чувствительные данные	_____	
5	Скрытые файлы	_____	
6	Разрешённые пути	_____	
7	Несуществующие пути	_____	
8	Общая статистика	_____	
9	Круговая диаграмма	_____	
10	Столбчатая диаграмма	_____	

Приложение А
УТВЕРЖДАЮ
Начальник тех.отдела
_____Корнеев Н.В.
“29” мая 2026

**ПРОГРАММА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПРИ
УГРОЗЕ ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА**

**ВЕДОМОСТЬ ЭКСПЛУАТАЦИОННЫХ
ДОКУМЕНТОВ**

Лист утверждения

XXXXXXX.XXXXXXX.ЭД1

Подпись и	
Инв. №	
Взам.	
Подпись и	
Инв. №	

Руководитель разработки
Корнеев Н.В.
“29” мая 2026

Ответственный исполнитель

Вотинцева А.С.
“29” мая 2026

УТВЕРЖДЕНО

XXXXXXX.XXXXXXX.ЭД1

**ПРОГРАММА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПРИ
УГРОЗЕ ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА**

**ВЕДОМОСТЬ ЭКСПЛУАТАЦИОННЫХ
ДОКУМЕНТОВ**

XXXXXXX.XXXXXXX.ЭД1

Инв. №	Подпись и	Взам.	Инв.	Подпись и

Обозначение: XXXXXX.XXXXXX.ЭД1

Наименование: Микросервис проверки запрещённых путей (Forbidden Paths Checker)

Разработчик: Вотинцева А.С.

Общие положения

Настоящая ведомость эксплуатационных документов содержит перечень эксплуатационных документов на программное изделие "Микросервис проверки запрещённых путей" и устанавливает их состав.

Ведомость составлена в соответствии с требованиями ГОСТ 19.507-79 "Единая система программной документации. Ведомость эксплуатационных документов. Требования к содержанию и оформлению".

Ведомость эксплуатационных документов

Таблица 25 - Ведомость эксплуатационных документов

Обозначение	Наименование	Примечание
XXXXXX.XXXXXX.ЭД1	Ведомость эксплуатационных документов	Настоящий документ
XXXXXX.XXXXXX.ЭД2	Формуляр	-
XXXXXX.XXXXXX.ЭД3	Описание применения	-

Сведения о документах

Ведомость эксплуатационных документов (XXXXXX.XXXXXX.ЭД1)

Назначение: Документ содержит перечень всех эксплуатационных документов на программное изделие.

Состав: перечень эксплуатационных документов с указанием обозначений и наименований.

Формуляр (XXXXXX.XXXXXX.ЭД2)

Назначение: Документ содержит основные характеристики программы, комплектность и сведения об эксплуатации.

Состав:

- общие указания;
- общие сведения;
- основные характеристики;

- комплектность;
- свидетельство о приёмке;
- сведения о рекламациях;
- сведения об изменениях.

Описание применения (XXXXXX.XXXXXX.ЭДЗ)

Назначение: Документ содержит сведения о назначении программы, области применения и методах использования.

Состав:

- назначение программы;
- условия применения;
- описание операций;
- возможные ошибки и способы их устранения;
- примеры использования.

Указания по пользованию документами

Таблица 26 - Указания по пользованию документами

Документ	Для кого предназначен	Когда используется
Ведомость эксплуатационных документов	Пользователь, программист	При ознакомлении с составом документации
Формуляр	Ответственный за эксплуатацию	При приёмке, учёте и контроле эксплуатации
Описание применения	Пользователь	При первом знакомстве с программой
Руководство программиста	Программист	При первом знакомстве с программой
Руководство оператора	Оператор	При первом знакомстве с программой

Приложение А
УТВЕРЖДАЮ
Начальник тех.отдела
_____Корнеев Н.В.
“29” мая 2026

**ПРОГРАММА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПРИ
УГРОЗЕ ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА**

Формуляр

Лист утверждения

XXXXXX.XXXXXX.ЭД2

Подпись и	
Инв. №	
Взам.	
Подпись и	
Инв. №	

Руководитель разработки
Корнеев Н.В.
“29” мая 2026

Ответственный исполнитель

Вотинцева А.С.
“29” мая 2026

УТВЕРЖДЕНО

XXXXXXX.XXXXXXX.ЭД2

**ПРОГРАММА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПРИ
УГРОЗЕ ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА**

Формуляр

XXXXXXX.XXXXXXX.ЭД2

Инв. №	Подпись и	Взам.	Инв.	Подпись и

1. Общие указания

Перед эксплуатацией программного изделия необходимо внимательно ознакомиться с «Руководством оператора» (XXXXXX.XXXXXX.ЭД3) и «Руководством программиста» (XXXXXX.XXXXXX.ЭД4). Формуляр должен находиться у лица, ответственного за эксплуатацию.

2. Общие сведения

Таблица 15 - Общие сведения

Параметр	Значение
Наименование	WAF-контейнер защиты от форсированного веб-браузинга
Обозначение	XXXXXX.XXXXXX
Разработчик	Вотинцева А.С.
Версия	1.0.0
Дата разработки	29.05.2026
Язык	Python 3.11 / Django 4.2
Назначение	Защита от УБИ.159 (форсированный веб-браузинг)

3. Основные характеристики

Таблица 16 - Основные характеристики

Характеристика	Значение
Макс. отслеживаемых IP	Не ограничено (ограничено ОЗУ)
Размер скользящего окна	60 секунд (настраиваемо)
Порог уникальных URL	50 (настраиваемо)
Порог ошибок 404	70% (настраиваемо)
Порог RPS	30 (настраиваемо)
Время блокировки	3600 сек = 1 час (настраиваемо)
Режимы	block / monitoring / disabled
Порт по умолчанию	8000
Дашборд	http://localhost:8000/waf/
Django Admin	http://localhost:8000/admin/ (admin/admin123)

4. Комплектность

Таблица 17 - Комплект документов

Обозначение	Наименование	Кол-во
XXXXXX.XXXXXX.001	Спецификация	1
XXXXXX.XXXXXX.002	Техническое задание	1
XXXXXX.XXXXXX.003	Текст программы	1
XXXXXX.XXXXXX.004	Описание программы	1
XXXXXX.XXXXXX.005	Пояснительная записка	1
XXXXXX.XXXXXX.006	Программа и методика испытаний	1
XXXXXX.XXXXXX.ЭД2	Формуляр	1
XXXXXX.XXXXXX.ЭД3	Руководство оператора	1
XXXXXX.XXXXXX.ЭД4	Руководство программиста	1

5. Периодический контроль основных характеристик

Таблица 18 - Контроль основных характеристик

Характеристика	Периодичность контроля	Метод
Доступность дашборда /waf/	Ежедневно	HTTP GET, ожидать 200 OK
Запись событий в JSON-лог	Еженедельно	Проверить logs/security_events.json
Количество активных блокировок	Ежедневно	GET /waf/api/stats/
Размер файла db.sqlite3	Ежемесячно	ls -lh db.sqlite3, при > 1 ГБ — очистка
Корректность порогов	При изменении трафика	Анализ статистики дашборда

6. Свидетельство о приёме

WAF-контейнер защиты от атаки форсированного веб-браузинга, обозначение XXXXXX.XXXXXX, прошёл испытания согласно «Программе и методике испытаний» (XXXXXX.XXXXXX.006) и признан годным к эксплуатации.

7. Гарантийные обязательства

Разработчик гарантирует соответствие программного изделия требованиям технического задания при соблюдении условий эксплуатации, изложенных в «Руководстве оператора».

Приложение А
УТВЕРЖДАЮ
Начальник тех.отдела
_____Корнеев Н.В.
“29” мая 2026

**ПРОГРАММА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПРИ
УГРОЗЕ ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА**

Описание применения

Лист утверждения

XXXXXX.XXXXXX.ЭДЗ

Подпись и	
Инв. №	
Взам.	
Подпись и	
Инв. №	

Руководитель разработки
Корнеев Н.В.
“29” мая 2026

Ответственный исполнитель
Вотинцева А.С.
“29” мая 2026

УТВЕРЖДЕНО
XXXXXXX.XXXXXXX.ЭДЗ

**ПРОГРАММА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПРИ
УГРОЗЕ ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА**

Описание применения
XXXXXXX.XXXXXXX.ЭДЗ

Инд. №	Подпись и	Взам.	Инд.	Подпись и

Обозначение: XXXXXX.XXXXXX.ЭДЗ

Наименование: Микросервис проверки запрещённых путей (Forbidden Paths Checker)

Разработчик: Вотинцева А.С.

Назначение программы

Программа предназначена для анализа входящих HTTP-запросов и выявления попыток несанкционированного доступа к защищённым, скрытым или системным ресурсам веб-приложения. Программа позволяет обнаруживать атаки типа "форсированный веб-браузинг" (Forced Browsing) и классифицировать их по категориям угроз.

Программа обеспечивает решение следующих задач:

1. Проверка URL-путей на наличие запрещённых паттернов.
2. Классификация угроз по пяти категориям (административные панели, системные файлы, бэкапы, чувствительные данные, скрытые файлы).
3. Сбор и визуализация статистики по проверенным запросам.
4. Отображение детальной таблицы результатов проверки.

Программа работает по следующему принципу: при обращении к веб-странице пользователя программа автоматически запускает проверку 32 тестовых запросов различных категорий. Каждый запрос анализируется на наличие запрещённых путей, после чего формируется статистика и отображаются графики.

Таблица 27 - Основные функциональные блоки

Блок	Назначение
Контейнер проверки	Содержит списки запрещённых путей и выполняет проверку
Статистика	Вычисляет общее количество, блокировки, процент блокировок
Визуализация	Отображает 4 графика и детальную таблицу результатов

Требования к подготовке пользователя

Для работы с программой пользователь должен обладать следующими навыками:

- запуск Django-сервера из командной строки;
- навигация по веб-интерфейсу;

– чтение результатов в табличном и графическом виде.

Специальной подготовки не требуется.

Описание операций

Установка программы

Шаг 1. Установка Python

Убедитесь, что на компьютере установлен Python версии 3.8 или выше.

Проверить версию можно командой:

```
bash python --version
```

Шаг 2. Установка Django

Установите Django с помощью pip:

```
bash pip install django
```

Шаг 3. Загрузка программы

Скопируйте файлы программы в рабочую директорию. Структура проекта должна включать:

Папка security - приложение с кодом микросервиса

Файл manage.py - управляющий скрипт Django

Папка templates - HTML-шаблоны

Шаг 4. Запуск сервера

Запустите Django-сервер командой:

```
bash python manage.py runserver
```

Запуск программы

Шаг 1. Открытие веб-браузера

Запустите любой современный веб-браузер (Chrome, Firefox, Edge, Safari).

Шаг 2. Ввод адреса

В адресной строке введите:

```
text http://127.0.0.1:8000
```

или

```
text http://localhost:8000
```

Шаг 3. Просмотр результатов

После загрузки страницы отобразится:

- карточки с общей статистикой;
- 4 интерактивных графика;
- детальная таблица результатов.

Интерпретация результатов

Таблица 28 - Карточки статистики

Показатель	Описание
Всего запросов	Общее количество проверенных запросов
Заблокировано	Количество запросов, которые были заблокированы
Разрешено	Количество запросов, которые были разрешены
Доля блокировок	Процент заблокированных запросов

Таблица 29 - Графики

График	Что отображает
Круговая диаграмма	Соотношение заблокированных и разрешённых запросов
Столбчатая диаграмма	Распределение запросов по категориям угроз
Кольцевая диаграмма	Распределение HTTP-кодов ответов
Горизонтальная диаграмма	Топ причин блокировки

Таблица 30 - Таблица результатов

Столбец	Описание
№	Порядковый номер запроса
Время	Время проверки (ЧЧ:ММ:СС)
Метод	HTTP-метод (GET или POST)
Путь	Проверяемый URL-путь
Код	HTTP-код ответа (200, 403, 404)
Статус	"Разрешено" или "Заблокировано"
Причина	Причина блокировки или разрешения

Таблица 31 - Возможные ошибки и способы их устранения

Ошибка	Причина	Способ устранения
ModuleNotFoundError: No module named 'django'	Django не установлен	Выполнить: <code>pip install django</code>
Port 8000 already in use	Порт занят другим приложением	Использовать другой порт: <code>python manage.py runserver 8080</code>
Страница не загружается	Сервер не запущен	Запустить сервер: <code>python manage.py runserver</code>
Графики не отображаются	Нет интернета для загрузки Chart.js	Подключиться к интернету или загрузить Chart.js локально
TemplateDoesNotExist	Отсутствует HTML-шаблон	Проверить наличие файла <code>index.html</code> в папке <code>templates</code>

Приложение А
УТВЕРЖДАЮ
Начальник тех.отдела
_____Корнеев Н.В.
“29” мая 2026

**ПРОГРАММА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПРИ
УГРОЗЕ ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА**

Руководство программиста

Лист утверждения

XXXXXX.XXXXXX.ЭД4

Подпись и	
Инв. №	
Взам.	
Подпись и	
Инв. №	

Руководитель разработки
Корнеев Н.В.
“29” мая 2026

Ответственный исполнитель
Вотинцева А.С.
“29” мая 2026

УТВЕРЖДЕНО

XXXXXXX.XXXXXXX.ЭД4

**ПРОГРАММА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПРИ
УГРОЗЕ ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА**

Руководство программиста

XXXXXXX.XXXXXXX.ЭД4

Инв. №	Подпись и	Взам.	Инв.	Подпись и

1. Назначение и условия применения программ

WAF-контейнер — Django-приложение, реализующее защиту от атак форсированного веб-браузинга. Программа работает как Django middleware в WSGI-окружении. Требуется Python 3.11+, Django 4.2+.

Условия применения:

- ОЗУ: не менее 256 МБ (рекомендуется 512 МБ при > 1000 активных IP).
- Потокное WSGI-окружение (gunicorn, uWSGI, встроенный dev-сервер Django).
- Запись в файловую систему: папки logs/ и sqlite-файл должны быть доступны.

2. Характеристика программы

Таблица 18 - Характеристика программы

Характеристика	Значение
Архитектура	Django Middleware + Singleton ProtectionEngine
Потокобезопасность	threading.Lock на каждый IPTracker
Хранение состояния	In-memory (dict + deque) + SQLite (персистентность)
Временная сложность check_and_record	O(n) по длине окна (deque.popleft)
Пространственная сложность	O(U × IP) где U — уникальных URL, IP — активных IP
Фоновая очистка	daemon-поток каждые 300 секунд
Автообновление дашборда	JavaScript fetch каждые 10 секунд

3. Обращение к программе

Точка входа в Django — settings.py, раздел MIDDLEWARE:

```
MIDDLEWARE = [
```

```
...
```

```
    'waf.middleware.ForcedBrowsingProtectionMiddleware',
]
```

Получение экземпляра движка из любого модуля:

```
from waf.protection import ProtectionEngine
engine = ProtectionEngine.get_instance()
```

4. Входные и выходные данные

Входные данные ProtectionEngine.check_and_record():

Таблица 19 - Входные данные

Параметр	Тип	Описание
ip	str	IP-адрес клиента (IPv4 или IPv6)
url	str	Полный URL запроса (включая query string)
path	str	Путь URL без query string
method	str	HTTP-метод: GET, POST, PUT и т.д.
status_code	int None	HTTP-статус ответа (None если заблокирован до backend)
user_agent	str	User-Agent заголовок клиента

Таблица 20 - Выходные данные

Поле	Тип	Значения / Описание
action	str	allow block monitor warn whitelist
reason	str None	unique_url_limit error_rate_limit rps_limit None
stats	dict	unique_urls, total_requests, error_404_count, error_rate

5. REST API — справочник эндпоинтов

Таблица 21 - Справочник эндпоинтов

Метод	URL	Описание	Тело / Параметры
GET	/waf/	HTML-дашборд	—
GET	/waf/api/stats/	Сводная статистика	—
GET	/waf/api/events/	Список событий	?limit=50&type=attack_detected
GET	/waf/api/config/	Текущий конфиг WAF	—
POST	/waf/api/block-ip/	Ручная блокировка	{"ip","duration","reason"}
POST	/waf/api/unblock-ip/	Разблокировка	{"ip"}
POST	/waf/api/simulate/	Симуляция атаки	{"ip","count"}
POST	/waf/api/clear-logs/	Очистка всех данных	—

6. Расширение и доработка

Добавление нового порогового критерия:

1. Добавить параметр в DEFAULT_CONFIG["thresholds"] в protection.py.
2. Добавить чтение параметра из waf_config.yaml.
3. Добавить условие проверки в метод check_and_record() после строки с rps_limit.
4. Добавить новое значение в BLOCK_REASON_CHOICES в models.py.
 - Выполнить: `python manage.py makemigrations waf && python manage.py migrate.`

- Замена SQLite на PostgreSQL:
- В settings.py заменить ENGINE на django.db.backends.postgresql.
- Указать DATABASE_URL в переменных окружения.
- Выполнить миграции на новой БД.
- Для in-memory счётчиков рассмотреть Redis + django-redis-cache.

7. Сообщения программисту

Таблица 22 - Сообщение программисту

Сообщение в логe	Причина	Действие
ProtectionEngine инициализирован	Штатный запуск	Ничего не требуется
Конфиг загружен из ...yaml	Конфиг прочитан	Ничего не требуется
Ошибка загрузки конфига	YAML не парсится	Проверить синтаксис waf_config.yaml
Ошибка блокировки IP	Ошибка записи в БД	Проверить доступность db.sqlite3
Ошибка записи лога запроса	Ошибка записи в БД	Проверить доступность db.sqlite3
Proxy error for ...	Backend недоступен	Проверить backend_url или оставить пустым

УТВЕРЖДАЮ
Начальник тех.отдела

_____Корнеев Н.В.

“29” мая 2026

**ПРОГРАММА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПРИ
УГРОЗЕ ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА**

Руководство оператора

Лист утверждения

XXXXXX.XXXXXX.ЭД5

Подпись и	
Инв. №	
Взам.	
Подпись и	
Инв. №	

Руководитель разработки
Начальник XXXX

_____Хxxxxxxxx Х.Х.

“ _____ ” _____200__

Ответственный исполнитель
Начальник гр. РиВ АСУТП
XXXX

_____Хxxxxxxxx Х.Х.

“ _____ ” _____200__

Исполнитель
Бед. инженер XXXX

_____Хxxxxxxxx Х.Х.

“ _____ ” _____200__

УТВЕРЖДЕНО

XXXXXXX.XXXXXXX.ЭД5

**ПРОГРАММА ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ ПРИ
УГРОЗЕ ФОРСИРОВАННОГО ВЕБ-БРАУЗИНГА**

Руководство оператора

XXXXXXX.XXXXXXX.ЭД5

Иув. №	Подпись и	Иув.	Взам.	Иув.	Подпись и

1. Назначение программы

WAF-контейнер предназначен для автоматической защиты веб-приложений от атак форсированного веб-браузинга. Программа перехватывает HTTP-запросы, анализирует активность IP-адресов и блокирует подозрительных клиентов.

2. Условия выполнения программы

Таблица 23 - Условия выполнения программы

Параметр	Требование
ОС	Linux / Windows / macOS
Python	3.11 и выше
ОЗУ	Не менее 256 МБ
Диск	Не менее 500 МБ
Сеть	Порт 8000 должен быть свободен
Зависимости	Django 4.2, requests, PyYAML, whitenoise

3. Выполнение программы

3.1. Установка и первый запуск

Переход в директорию: `cd artifacts/waf-container`

Установка зависимостей: `pip install -r requirements.txt`

Применение миграций: `python manage.py migrate`

Загрузка демо-данных: `python manage.py seed_demo`

Запуск сервера: `python manage.py runserver 0.0.0.0:8000`

3.2. Работа с дашбордом

После запуска открыть в браузере: `http://localhost:8000/waf/`

Дашборд содержит следующие разделы:

- карточки KPI: всего запросов, заблокировано, активных блокировок, событий;
- график трафика по часам (Chart.js, обновляется каждые 10 сек);
- круговая диаграмма причин блокировок;
- таблица последних событий безопасности;
- таблица активных блокировок с кнопкой «Разблокировать»;
- кнопка «Симулировать атаку» для демонстрации работы WAF;

3.3. Управление режимом работы

Режим работы изменяется в файле `config/waf_config.yaml`, параметр `mode`. После изменения требуется перезапуск сервера.

Таблица 24 - Управление режимом работы

Режим	Значение в YAML	Поведение
Блокировка	<code>mode: block</code>	IP блокируется при атаке (режим <code>production</code>)
Мониторинг	<code>mode: monitoring</code>	Атака фиксируется, трафик не блокируется
Отключён	<code>mode: disabled</code>	WAF прозрачен, все запросы пропускаются

3.4. Управление блокировками

Ручная блокировка через дашборд:

На дашборде `/waf/` в разделе «Управление» ввести IP и нажать «Заблокировать».

Ручная блокировка через API:

```
curl -X POST http://localhost:8000/waf/api/block-ip/ \
  -H "Content-Type: application/json" \
  -d '{"ip": "1.2.3.4", "duration": 3600}'
```

3.5 Настройка пороговых значений

Откройте файл `config/waf_config.yaml` и измените нужные параметры. Перезапустите сервер для применения изменений.

Таблица 25 - Настройка пороговых значений

Параметр	По умолчанию	Рекомендуемый диапазон
<code>unique_urls</code>	50	20–100 (меньше = чувствительнее)
<code>error_rate_percent</code>	70	50–90
<code>rps</code>	30	10–100
<code>window_seconds</code>	60	30–300
<code>block_duration_seconds</code>	3600	600–86400

4. Сообщения оператору

Сообщение / ситуация	Причина	Действие оператора
Страница <code>/waf/</code> недоступна (ERR_CONNECTION)	Сервер не запущен	Выполнить: <code>python manage.py runserver 0.0.0.0:8000</code>
HTTP 500 при открытии <code>/waf/</code>	Ошибка миграций	Выполнить: <code>python manage.py migrate</code>
Активных блокировок 0 после симуляции	Режим <code>monitoring</code> или <code>disabled</code>	Проверить <code>mode</code> в <code>waf_config.yaml</code>
IP не блокируется несмотря на атаку	IP в <code>whitelist</code>	Проверить раздел <code>whitelist</code> в <code>waf_config.yaml</code>
Bad Gateway 502	Backend недоступен	Проверить <code>backend_url</code> в <code>waf_config.yaml</code> или оставить пустым для демо-режима
Лог не пишется в <code>security_events.json</code>	Нет прав на папку <code>logs/</code>	Выполнить: <code>mkdir -p logs && chmod 755 logs</code>